



DATA MANAGEMENT 2 • FINAL PROJECT

Real-Time Weather and Air Quality Data Pipeline

Open-Meteo • Spark • BigQuery • dbt • Airflow

Devendra Singh Dhakad • Matriculation No. 100004684

Data Management 2: Data Curation and Data Management

Examiner: Prof. Binh Vu

SRH University

June 2026



1 Project goal

- Build a complete, automated data pipeline that collects live weather and air quality for ten European cities, cleans and models the data, and serves analytics-ready tables in BigQuery.
- It demonstrates the full data-curation lifecycle: real-time extraction, distributed cleaning, warehouse loading, SQL transformation with tests, and hands-off scheduling.

2

data sources
(one real-time)

10

European cities
polled live

5 min

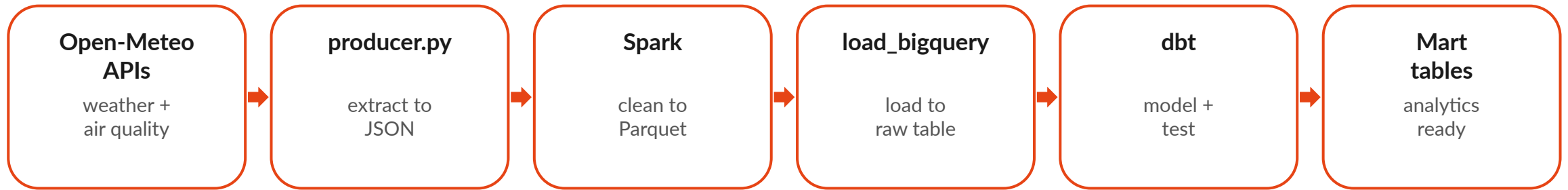
automatic
run interval

6 / 6

exam requirements
met



2 Architecture and data flow



Apache Airflow runs every step, every 5 minutes (cron fallback: run_pipeline.sh)

Two isolated Python 3.11 environments keep dependencies apart: .venv-pipeline (Spark, dbt, BigQuery) and airflow/.venv-airflow (Airflow 2.10). Airflow tasks shell into the pipeline environment.



3 Data sources: two, one real-time

Source 1: Open-Meteo (real-time)

Two free REST APIs, no key. Weather (temperature, humidity, wind, code) and air quality (PM10, PM2.5, European AQI, ozone).

Polled for 10 cities every 5 minutes, so the landing zone is a live, growing stream of observations.

Source 2: City reference (static)

A curated CSV of the 10 cities (country, country name, latitude, longitude, population, timezone), loaded as a dbt seed.

Reference data that rarely changes. It enriches every reading and anchors a referential-integrity test.

The two sources join on city: a fast-moving stream combined with slow-moving reference data, a classic curation pattern.



4 Extract: producer.py

- Calls both Open-Meteo endpoints for each city and writes one newline-delimited JSON file per run into the landing zone.
- `timezone=UTC` keeps every observation timestamp comparable.
- Per-city try and except: one failed city does not abort the whole run.
- The nesting is deliberate, so Spark does the real flattening downstream.

data/landing/readings_.json (one line per city)*

```
{
  "city": "Berlin", "country": "DE",
  "weather": {
    "time": "2026-06-14T15:45",
    "temperature_2m": 15.9,
    "relative_humidity_2m": 64,
    "wind_speed_10m": 20.8 },
  "air_quality": {
    "pm2_5": 2.4, "european_aqi": 26 },
  "ingested_at": "2026-06-14T15:50:19Z"
}
```



5 Clean with Spark

- 1 Flatten the nested weather and air-quality blocks
- 2 Cast every field to its proper type
- 3 Parse the time string into a real timestamp
- 4 Drop rows with null city, time or temperature
- 5 Deduplicate by (city, observed_at) in the batch
- 6 Derive aqi_category from the European AQI bands

Spark Structured Streaming, Trigger.AvailableNow

```
cleaned = (batch_df.select(  
    F.col('city'),  
    F.to_timestamp(F.col('weather.time'),  
        "yyyy-MM-dd'T'HH:mm")  
        .alias('observed_at'),  
    F.col('weather.temperature_2m'))  
    .where(F.col('temperature_2m').isNotNull())  
    .dropDuplicates(['city', 'observed_at']))
```



6 Load into BigQuery

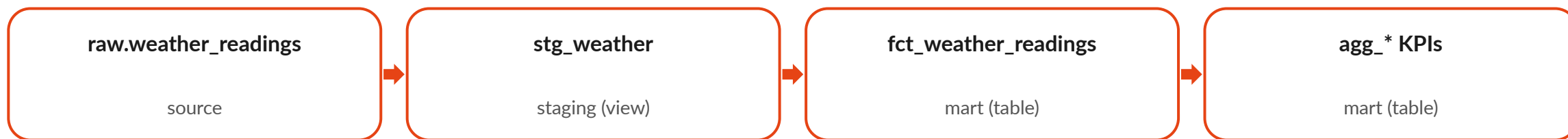
- `load_bigquery.py` reads all cleaned Parquet and loads it into `raw.weather_readings` with `WRITE_TRUNCATE`.
- Idempotent: running it twice gives the same table, no duplicates, so Airflow can safely retry.
- Uses the `google-cloud-bigquery` client only. No `gcloud` CLI; auth via a service account key.
- Verified: 30 rows loaded into the raw table.

`raw.weather_readings`

<code>city, country</code>	<code>STRING</code>
<code>latitude, longitude</code>	<code>FLOAT</code>
<code>observed_at</code>	<code>TIMESTAMP</code>
<code>temperature_2m</code>	<code>FLOAT</code>
<code>relative_humidity_2m</code>	<code>INTEGER</code>
<code>wind_speed_10m</code>	<code>FLOAT</code>
<code>pm10, pm2_5, ozone</code>	<code>FLOAT</code>
<code>european_aqi</code>	<code>INTEGER</code>
<code>aqi_category</code>	<code>STRING</code>
<code>ingested_at</code>	<code>TIMESTAMP</code>



7 Transform with dbt (ELT)



The static seed (stg_cities) joins into fct_weather_readings to add country name and population.

Why ELT, not ETL

Spark does only light cleaning. All business logic then runs as dbt SQL inside BigQuery: warehouse scale, version-controlled SQL, lineage, and built-in tests.

The models

```
staging:  stg_weather, stg_cities
marts:    fct_weather_readings
          agg_weather_by_city
          agg_air_quality_by_country
```

5 models built successfully.



8 Data quality: 17 tests, all pass

not_null

Key fields are never empty: city, observed_at, reading_id.

unique

No duplicate keys: reading_id and city are unique in the marts.

accepted_values

aqi_category is always one of the six European AQI bands.

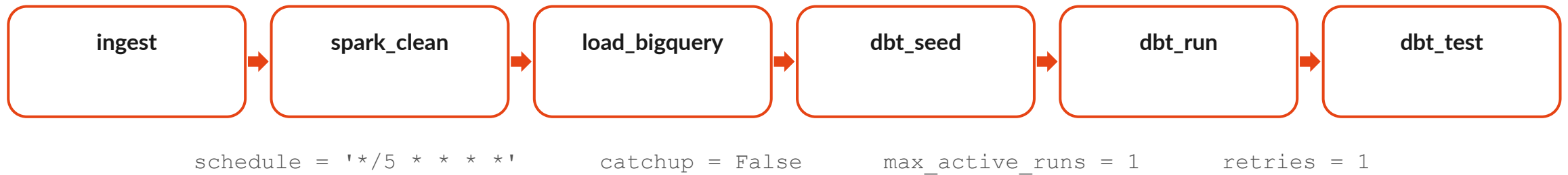
relationships

Every reading's city exists in the static city reference.

dbt test fails the Airflow task if any check fails, so bad data is caught before anyone trusts the KPIs.



9 Automation: Apache Airflow



Verified run: the DAG executed end to end through Airflow. DagRun state = success; all six tasks (ingest, spark_clean, load_bigquery, dbt_seed, dbt_run, dbt_test) returned success.

dbt_seed loaded the cities, dbt_run built 5 models, dbt_test passed all 17 checks against BigQuery.

run_pipeline.sh runs the same six-step chain and can be scheduled with cron as a fallback.

10 Sample results from the marts

Air quality by country (average European AQI, lower is better)

Country	Avg PM2.5	Avg EAQI
Italy	4.7	45.0
Portugal	9.7	41.0
Spain	6.1	40.0
France	4.4	34.0
Germany	3.0	29.5
Poland	4.3	26.5

Highlights

Hottest city: Madrid, 31.8 C average

Coollest city: Warsaw, 13.25 C average

Worst air quality: Italy (EAQI 45)

Best air quality: Poland (EAQI 26.5)

Counts grow every 5 minutes as new hourly observations arrive, so the averages become richer over time.



11 How each requirement is met

Requirement	Where it is implemented
1. Two sources, one real-time	Open-Meteo APIs (real-time) + city reference seed (static)
2. Extract, clean, load to BigQuery	producer.py, spark_clean.py, load_bigquery.py
3. Transformation with dbt	seed + staging + marts, all SQL inside BigQuery (ELT)
4. Include Spark	Spark Structured Streaming cleaning job
5. Runs automatically	Airflow DAG every 5 min (cron fallback: run_pipeline.sh)
6. Presentation	this deck: sources, cleaning, transformation explained